Software Engineering Project

Requirement Specification & Design Document

Hotel Management System

Prepared by: Group03

Team Members: Nguyen Cong Thanh (Leader)
Luong Nhat Khoi (Vice Leader)
Huynh Nhat Hoa (Member)
Nguyen Hoang Bao Anh (Member)
Nguyen Minh Man (Member)

Course: Software Engineering

This sample document can be edited to match the final team submission.

Contents

1 Introduction **2**
1.1 Objectives 2

2 Requirement Specification **2**
2.1 Functional Requirements 2
2.2 Non-Functional Requirements 3

3 System Modeling **3**
3.1 Data Flow Diagram 3
3.2 Use Case Diagram 4
3.2.1 Actor List 6
3.2.2 Use-case List 6
3.2.3 Detailed Use-case Specifications 7
3.3 Class Diagram 11

4 System Design **12**
4.1 Data Model 12
4.1.1 Key Constraints 13
4.2 Interface Design Description 13

5 Implementation Overview **14**
5.1 Technology Stack 14
5.2 Module Organization 15

6 Testing and Bug Management **15**
6.1 Testing Strategy 15
6.2 Sample Test Cases 15
6.3 Debugging and Bug Management 16

7 Conclusion **17**

1. Introduction

The Hotel Management System is a web-based application that supports daily hotel operations such as authentication, room management, booking processing, customer management, payment tracking, and reporting. This document presents a concise requirement specification and design sample based on the current project and aligned with the course rubric.

1.1. Objectives

- Support the main workflows for staff and customers.
- Define the core models, interface, and database structure.
- Summarize implementation and testing at a report level.

2. Requirement Specification

2.1. Functional Requirements

The main actors are Admin, Staff, and Customer. The most important functional requirements are summarized in Table 1.

Module	Key Requirements
Authentication	Users can register, log in, log out, and recover passwords. The system enforces role-based access.
Rooms	Customers can browse rooms. Staff can add, update, delete, and change room status.
Bookings	Customers can request bookings. Staff can create, approve, cancel, check in, and check out bookings. The system validates room availability and total cost.
Customers	Staff can create, search, update, delete, and export customer records.
Payments and Reports	The system records payments and provides dashboard metrics and date-based reports for staff.

Table 1: Core functional requirements

2.2. Non-Functional Requirements

Aspect	Requirement
Usability	The interface should be easy to learn, with clear forms and validation messages.
Security	Passwords must be hashed and access control must follow user roles.
Reliability	Foreign keys and business rules must prevent invalid bookings and inconsistent data.
Performance	Common operations such as search, filtering, and form submission should be responsive.
Maintainability	The project should remain modular, with separate controllers, models, templates, and database utilities.
Constraints	The current implementation uses Python, a web interface, and SQLite.

Table 2: Key non-functional requirements

3. System Modeling

3.1. Data Flow Diagram

The DFD shows how staff and customers interact with the system and how data moves between the main business processes.

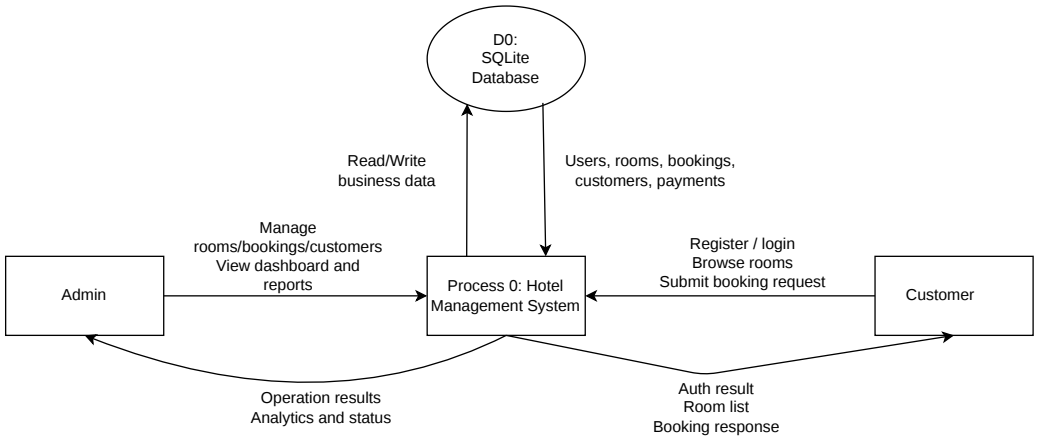

Figure 1: Context-level Data Flow Diagram

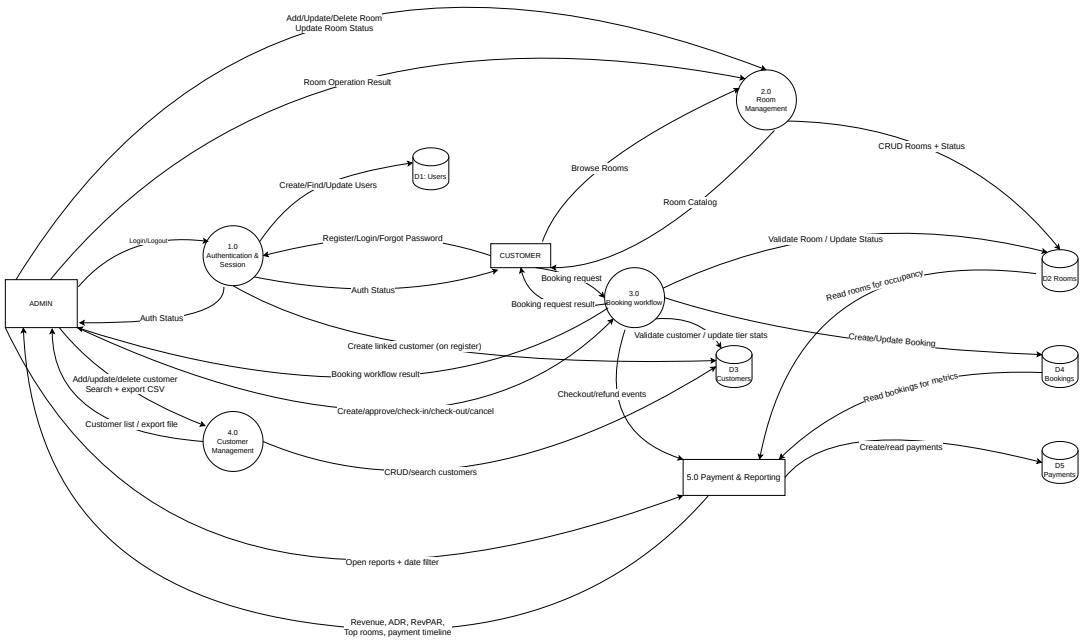

Figure 2: Level 1 Data Flow Diagram

At the context level, the system communicates with staff and customers. At Level 1, it is divided into authentication, room management, booking workflow, customer management, and payment/reporting.

3.2. Use Case Diagram

The Use Case Diagram summarizes the main interactions between actors and system functions.

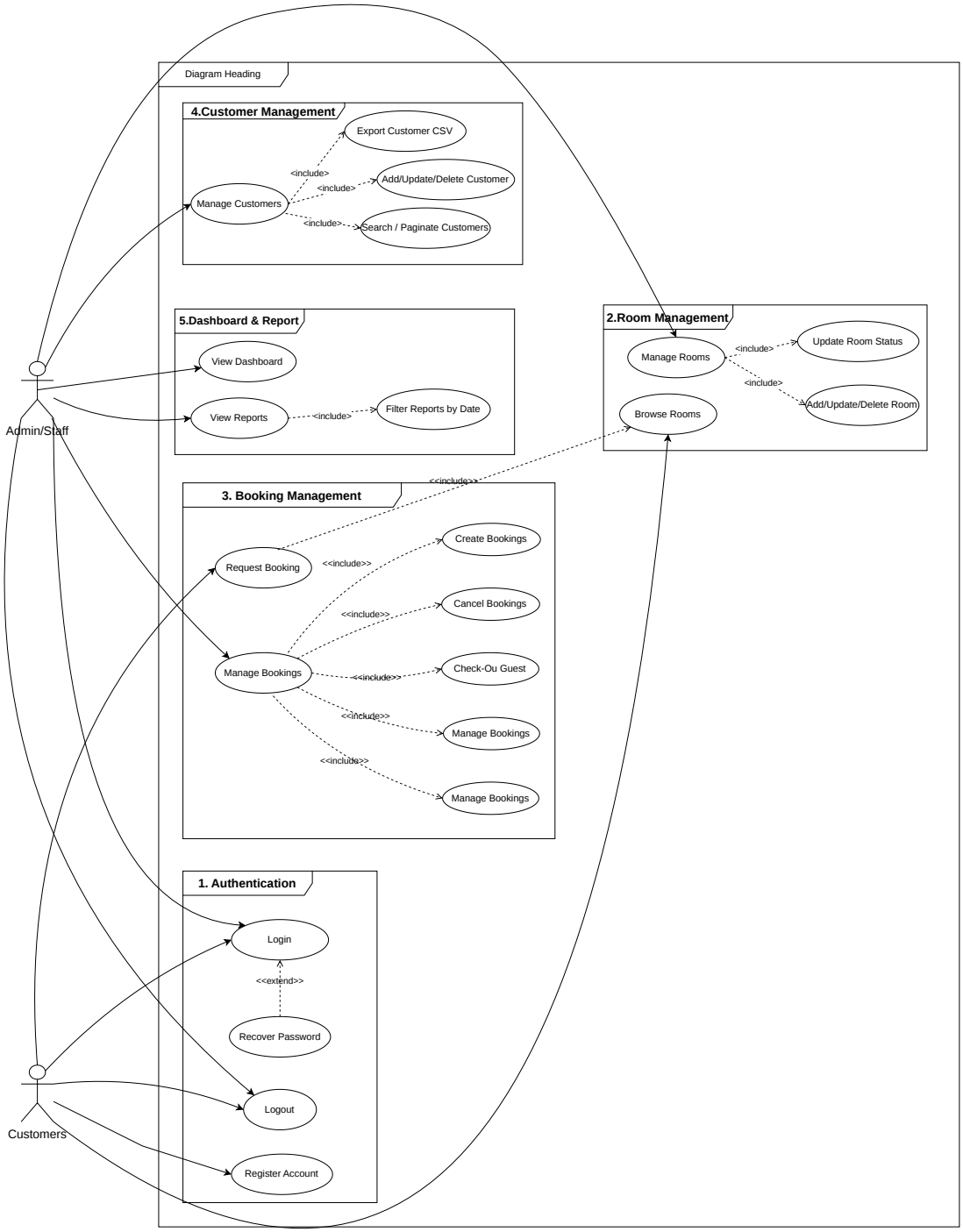

Figure 3: Use Case Diagram of the Hotel Management System

3.2.1. Actor List

Actor Name	Meaning
Admin	A privileged hotel user who supervises operations, views reports, and manages rooms, customers, and bookings.
Staff	A hotel employee who performs daily operational tasks such as room updates, customer handling, booking processing, and report viewing.
Customer	An end user who registers an account, logs in, browses rooms, and submits booking requests.

Table 3: Actor List

3.2.2. Use-case List

Use-case	Meaning
Login	Allows a customer or hotel staff member to access the system using valid credentials.
Logout	Ends the current user session securely.
Register Account	Allows a new customer to create an account in the system.
Recover Password	Allows a customer to reset or recover access to an account when the password is forgotten.
View Dashboard	Allows Admin or Staff to view occupancy, booking, and revenue summaries.
Manage Rooms	Allows Admin or Staff to oversee room information and room status.
Manage Bookings	Allows Admin or Staff to process booking operations and booking status changes.
Manage Customers	Allows Admin or Staff to maintain customer information.
View Reports	Allows Admin or Staff to review date-based operational and financial reports.
Browse Rooms	Allows customers to view available room information before booking.
Request Booking	Allows a customer to submit a booking request for a selected room and date range.

Use-case	Meaning
Add/Update/Delete Room	Allows staff to create, edit, or remove room records.
Update Room Status	Allows staff to change room states such as available, occupied, cleaning, or maintenance.
Create Booking	Allows staff to create a booking directly for a customer.
Approve Booking	Allows staff to confirm a pending booking request.
Check-In Guest	Allows staff to mark a confirmed booking as checked in.
Check-Out Guest	Allows staff to complete a stay and finalize checkout actions.
Cancel Booking	Allows staff to cancel a booking according to business rules.
Search/Paginate Customers	Allows staff to search customer records and browse them page by page.
Add/Update/Delete Customer	Allows staff to maintain customer records through CRUD operations.
Export Customers CSV	Allows staff to export customer data for reporting or backup purposes.
Filter Reports by Date	Allows staff to limit reports to a selected date range.

Table 4: Use-case List

3.2.3. Detailed Use-case Specifications

Use Case 1: Login

Brief Summary: This use case allows a customer, staff member, or administrator to authenticate with the system and access the appropriate interface based on role.

Main Flow of Events:

1. The use case begins when the user opens the login page.
2. The system displays fields for email and password.
3. The user enters valid credentials and submits the form.
4. The system validates the credentials against stored account data.
5. The system identifies the role of the authenticated user.

6. The system creates a session and redirects the user to the corresponding dashboard or user area.

Alternative/Exception Flows:

- If the email or password is incorrect, the system displays an error message and returns to the login form.
- If required fields are empty, the system asks the user to complete the missing information.
- If the user cannot remember the password, the user may move to the Recover Password use case.

Special Requirements:

- Passwords must be verified using secure hashing.
- The system must enforce role-based access after login.
- Validation messages should be clear and user-friendly.

Pre-conditions:

- The user already has a valid account in the system.
- The login page is available.

Post-conditions:

- If successful, the user is authenticated and an active session is created.
- If unsuccessful, no session is created and the user remains on the login page.

Extension Points:

- Recover Password.
- Redirect to dashboard or customer interface based on user role.

Use Case 2: Manage Rooms

Brief Summary: This use case allows Admin or Staff to maintain room records and room status information so that room availability is always accurate.

Main Flow of Events:

1. The use case begins when Admin or Staff opens the room management screen.

2. The system displays the current room list with relevant details such as room number, type, price, and status.
3. The user selects one of the available actions: add room, update room, delete room, or update room status.
4. If the user adds or updates a room, the system displays a form for room information.
5. The user enters or modifies the required room data and confirms the action.
6. The system validates the input, saves the changes, and refreshes the room list.
7. The system shows a success notification for the completed action.

Alternative/Exception Flows:

- If required room information is missing or invalid, the system displays validation errors and asks the user to correct the form.
- If the room number already exists, the system rejects the action and informs the user about the duplicate value.
- If the user attempts to delete a room linked to an active booking, the system blocks the deletion or applies the project's business rule safely.
- If the user cancels the action, the system returns to the room list without saving changes.

Special Requirements:

- Room numbers must remain unique.
- Room status must follow predefined values such as available, occupied, cleaning, or maintenance.
- Only authorized Admin or Staff users may access this use case.

Pre-conditions:

- The user is logged in as Admin or Staff.
- The room management module is available.

Post-conditions:

- If successful, the room record or room status is updated in the database.
- If unsuccessful, the original room data remains unchanged.

Extension Points:

- Add / Update / Delete Room.
- Update Room Status.

Use Case 3: Request Booking

Brief Summary: This use case allows a customer to request a room booking by selecting a room, choosing dates, and submitting reservation details to the system.

Main Flow of Events:

1. The use case begins when a customer browses available rooms and selects a room to book.
2. The system displays room details and a booking request form.
3. The customer enters booking information such as check-in date, check-out date, and number of guests.
4. The customer submits the booking request.
5. The system validates the entered dates, guest information, and room availability.
6. The system calculates the total booking amount.
7. The system creates a booking request with an initial status such as pending.
8. The system confirms that the booking request has been recorded.

Alternative/Exception Flows:

- If the selected room is not available for the requested date range, the system rejects the request and shows an availability message.
- If the customer enters invalid dates or incomplete information, the system displays validation errors and requests correction.
- If the customer is not logged in, the system redirects the user to the login process before completing the request.
- If a technical error occurs during submission, the system does not create the booking and informs the customer that the request failed.

Special Requirements:

- The system must prevent overlapping bookings for the same room.
- Booking totals must be calculated using room price and stay duration.
- The customer must only be able to request rooms that are available for booking.

Pre-conditions:

- The customer has access to the room browsing module.
- Room information is available in the system.

Post-conditions:

- If successful, a new booking request is stored with the correct customer, room, and date information.
- If unsuccessful, no booking record is created.

Extension Points:

- Browse Rooms.
- Manage Bookings (for later approval or cancellation by staff).

Use Case 4: Check-out and Payment

Brief Summary: This use case describes the process where a staff member finalizes a customer's stay, processes the payment, and updates room availability.

Main Flow of Events:

1. The staff member opens the booking details for a guest currently `checked_in`.
2. The system calculates the final balance based on stay duration and additional services.
3. The staff member selects the payment method and records the transaction.
4. The system validates the payment and creates a record in the `payments` table.
5. The system updates the booking status to `completed`.
6. The system triggers the room status update to `cleaning`.

Alternative/Exception Flows:

- If the payment transaction fails, the system logs the error and maintains the booking as `checked_in` with a "payment pending" flag.

3.3. Class Diagram

The Class Diagram presents the high-level software structure and relationships between entities, models, and controllers.

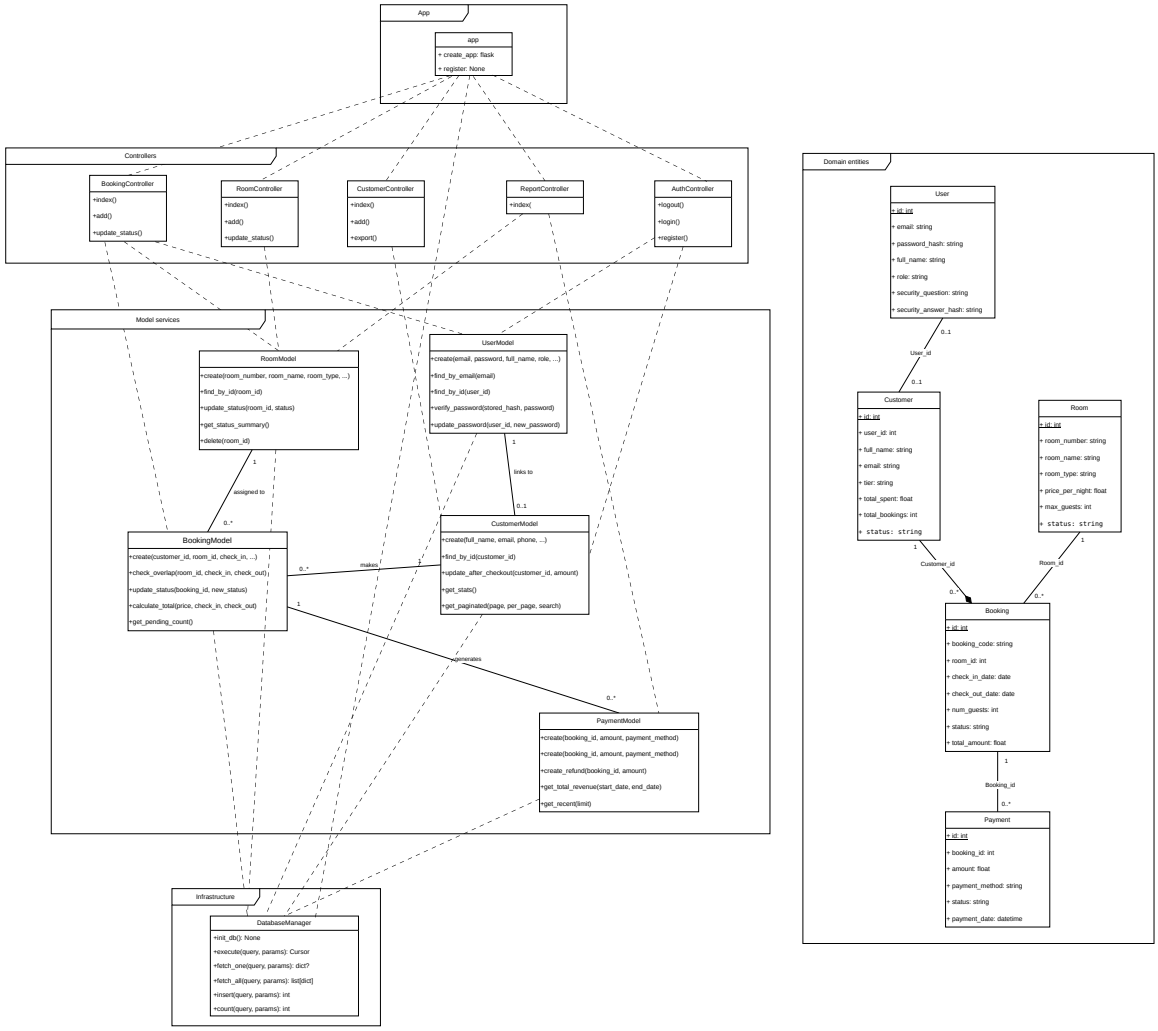

Figure 4: Class Diagram of the Hotel Management System

The design is modular. Core entities include User, Room, Customer, Booking, and Payment. Controllers handle requests, model classes manage business rules, and templates provide the interface.

4. System Design

4.1. Data Model

The database uses a relational schema with five main tables.

Table	Primary Key	Foreign Keys	Purpose
users	id	None	Stores credentials (email, hashed password) and roles for system access.

Table	Primary Key	Foreign Keys	Purpose
customers	id	user_id references users(id)	Stores profile details (name, phone, loyalty tier) linked to a unique user account.
rooms	id	None	Stores room inventory, categories, and real-time status.
bookings	id	customer_id references customers(id); room_id references rooms(id)	Tracks reservation lifecycle from request to completion.
payments	id	booking_id references bookings(id)	Records financial transactions, linking payments directly to a specific stay.

Table 5: Main database tables (Normalized)

4.1.1. Key Constraints

- **Identity Uniqueness:** `users.email` acts as the primary identifier and must be unique. The `customers` table inherits identity via `user_id`.
- **Operational Integrity:** `check_out_date` must be strictly later than `check_in_date`.
- **Payment-Booking Linkage:** A `payment` record can only be created if a `booking` status is `checked_out` or `confirmed` (for deposits).
- Booking status should be limited to values such as `pending`, `confirmed`, `checked_in`, `checked_out`, and `cancelled`.
- Room status should be limited to operational values such as `available`, `occupied`, `cleaning`, and `maintenance`.

4.2. Interface Design Description

The user interface is web-based and organized around role-specific workflows.

Screen	User	Description
Login Page	All users	Handles authentication and access control.
Dashboard	Admin / Staff	Shows room, booking, occupancy, and revenue summaries.
Room and Customer Management	Admin / Staff	Supports CRUD, search, pagination and status updates.
Booking Management	Admin / Staff	Supports booking creation and status changes such as approve, check-in, and check-out.
Room Browsing and Booking	Customer	Shows room details and allows customers to submit booking requests.
Reports View	Admin / Staff	Displays date-filtered financial and operational reports.

Table 6: Main interface screens

The interface should emphasize clear navigation, visible status indicators, and consistent forms. Tables should support search and filtering where necessary, and important actions such as deleting a customer or cancelling a booking should require confirmation.

5. Implementation Overview

5.1. Technology Stack

Layer	Technology / Tool
Programming Language	Python 3.x
Backend Framework	**Flask** (Handling routes, business logic, and session management)
Template Engine	**Jinja2** (Dynamic HTML rendering and data binding)
Database	SQLite (Relational storage via SQLAlchemy ORM or raw SQL)
Frontend	HTML5, CSS3 (Structured via Google-Stitch methodology)
Version Control	Git with GitHub or GitLab
Testing and Support Tools	Pytest, Excel, Jira, Bugzilla, Mantis, Docker, diagrams.net, and Plant UML

Table 7: Implementation stack reflecting the Web Architecture

5.2. Module Organization

The current codebase is organized into multiple layers:

- `src/controllers/`: request-handling logic for authentication, dashboard, room, booking, customer, and reporting features.
- `src/models/`: business logic and data access for users, rooms, customers, bookings, and payments.
- `src/database/`: schema definitions and database manager utilities.
- `src/templates/`: HTML templates for the interface.
- `test/`: unit and functional test files.

This structure separates interface logic, business rules, and persistence, which makes maintenance and teamwork easier.

6. Testing and Bug Management

6.1. Testing Strategy

Testing is performed at unit, integration, and manual levels:

- **Unit testing:** validate core model logic such as booking overlap checks, customer validation, and room status updates.
- **Integration testing:** confirm that controllers, models, templates, and database operations work together correctly.
- **Manual testing:** verify complete user flows such as login, create booking, approve booking, and generate reports.
- **Regression testing:** re-run important scenarios after bug fixes or feature changes.

6.2. Sample Test Cases

Test ID	Scenario	Expected Result
TC-01	Valid staff login with correct email and password	The system authenticates the user and opens the staff dashboard.

Test ID	Scenario	Expected Result
TC-02	Customer registration with missing required fields	The system rejects the form and displays validation messages.
TC-03	Create a booking for a room that is already reserved in the same date range	The system prevents the booking and shows an availability error.
TC-04	Update a room status from available to maintenance	The room record is updated and no longer appears as bookable.
TC-05	Delete a customer with active bookings	The system blocks deletion or enforces the defined business rule safely.
TC-06	Record a completed payment for a confirmed booking	A payment record is created and linked to the booking.
TC-07	Filter reports by date range	The report view only displays metrics and payment data inside the selected interval.
TC-08	Customer submits a valid booking request	The booking is stored with the correct customer, room, date range, and initial status.

Table 8: Sample Test Cases

6.3. Debugging and Bug Management

When a defect is found, the team should record the issue, reproduce it consistently, isolate the cause, implement a fix, and verify the result. A simple bug management workflow may include the following states:

- **Open:** the bug is reported and awaiting triage.
- **In Progress:** a team member is reproducing or fixing the issue.
- **Resolved:** the fix has been implemented and locally verified.
- **Closed:** the team confirms the bug no longer occurs.

For class projects, bugs can be tracked in Excel if the team needs a lightweight solution. If the project becomes more complex, tools such as Jira, Bugzilla, or Mantis Bug Tracker provide better history, ownership, and workflow visibility.

7. Conclusion

This sample report presents the essential parts of a requirement specification and design document for the Hotel Management System. It can be extended later with screenshots, more detailed use-case descriptions, or additional test evidence if needed.